



FRI Course Assistant: A Specialized Chatbot for Academic Course Selection

Strahinja Dorđević, Stefan Anđelković

Abstract

This paper presents a chat assistant designed to help students choose courses at the Faculty of Computer and Information Science, University of Ljubljana. The system is built on a Retrieval-Augmented Generation (RAG) architecture combined with a Large Language Model, and includes an agentic component with tools for timetable querying and overlap detection. The knowledge base is derived from faculty course catalogues, with a custom chunking strategy designed to preserve course-level context during retrieval. To evaluate the system, we created a benchmark of queries across four categories: simple factual, comparison, timetable, and complex queries. Evaluation was conducted in both Slovenian and English using three metrics: LLM-based scoring, cosine distance, and human evaluation. Results were compared against DeepSeek as a baseline. The system demonstrates strong performance on factual and timetable queries, with results comparable to larger general-purpose language models.

Keywords

Retrieval-Augmented Generation, course recommendation system, educational chatbot, document chunking, agentic system, timetable reasoning, multilingual NLP

Advisors: Slavko Žitnik

Introduction

Since the popularization of large language models, there have been many solutions developed to solve different every day problems. One such example are chat bots, the most popular being ChatGPT. Many solutions take a pretrained model and finetune them for specialized use-cases, usually in the form of an assistant that the user can ask question to (e.g. an online shopping website, where you can ask questions about products). This paper will present one such use-case.

At the Faculty of computer and information science, University of Ljubljana (slo. FRI), a common problem students face is picking elective courses. This seems to be an especially relevant problem at FRI because it has so many different courses. For this reason, we propose a chat assistant that students can ask questions about courses, to help them decide which courses to pick. Students can say what topics interest them, how many ECTS credits they need, etc., and the chatbot will give them suggestions for courses that best meet their needs. We also implement a mechanism that will let the student give specific time slot requirements.

Related work

Specialized AI assistants have emerged across various industries. In finance, Bank of America's Erica helps nearly 50 million users with spending management and financial guidance. In retail, Bindly offers personalized shopping recommendations through natural conversation, while H&M's chatbot provides outfit and style advice on messaging platforms. In the culinary domain, the Cookidoo Assistant integrated into Bimby TM7 helps users discover recipes based on available ingredients and can generate new preparations tailored to dietary constraints.

In recent years, the use of chatbots has grown rapidly across various fields, including education. Ranoliya et al. [1] developed a chatbot for university-related FAQs, while Clarizia et al. [2] created an educational chatbot prototype employing natural language processing and domain ontologies to effectively answer student queries. Most notably, a recent comprehensive survey by Swacha and Gracel [3] analyzed 47 papers on Retrieval-Augmented Generation (RAG) chatbots in education, examining their purposes, thematic scope, and evaluation methods.

At the Faculty of Computer and Information Science, stu-

dents face difficulty selecting elective courses due to the large number of options. We propose a chat assistant that helps students choose courses based on their interests, ECTS requirements, and scheduling constraints. Such a chatbot would greatly benefit the FRI faculty by reducing the administrative burden on staff, who currently spend significant time answering individual student inquiries about course selection, while simultaneously helping students make more informed decisions about their academic paths.

Dataset

The main source of information our chatbot will use are PDF files containing in-depth information about every course that the Faculty offers [4, 5]. These contain general information about each course, the number of credits they give, the semester and year they are held in, the field of study the course is meant for, as well as some other details. We will also try to obtain information about the time table for the current year [6]. We will also include all generally available data on the FRI website [7].

Methods

Document Processing

The knowledge base for our chatbot is built from PDF course catalogues for the University of Ljubljana, Faculty of Computer and Information Science. For conversion of documents to markdown format, we have tried many converters, but the best results were achieved using the `Docling` library which uses a multimodal approach. This is important because our input data includes a lot of tables and accurate table extraction is important for the quality context to be passed into the models. After conversion, the markdown still required some manual processing as the original PDF documents didn't have a uniform format for all courses and this was needed for chunking.

Chunking

Perhaps the biggest challenge for this project was how to chunk the input documents so that the LLM would receive relevant and complete information for the input question. Using a standard recursive splitter method for chunking was insufficient because the context would mostly include random parts of course description with no information about what course those descriptions belonged to. This led to the LLM model often making up course names that don't exist, since it wasn't given proper information.

Next, we tried making each chunk one course. This way, every chunk contained all necessary information for the LLM to answer the question. This approach had multiple problems. Since some chunks were very big, LLMs required a much bigger attention matrix to process them. This, combined with the fact that our model is agentic, meaning it takes multiple turns to answer the question, meant the model was simply using too much VRAM. For the queries it was able to process, the provided context was now so big that the LLMs could

not retrieve relevant answers. The final and best chunking methods combined concepts from the previous methods. First, the documents are split by course. Then, each course chunk is split using a recursive splitter but for each chunk we save as metadata the course from which the chunk was extracted. During retrieval, only the portion of each course description that is relevant to the query is returned as a chunk. In addition, the name of the course to which each chunk belongs is appended to the context to preserve identification and improve interpretability. For recursive splitting, we use a chunk size of 1500 characters with an overlap of 200.

Retrieval

Chunks are embedded using the `sentence-transformers/all-MiniLM-L6-v2` sentence transformer. Since course information can be both in English and in Slovenian it is important that the embedding model is multilingual. We also tried using larger embedding models but they were not needed and the MiniLM retrieved accurate results.

At query time, the user's question is first processed by the reasoning model `Qwen2.5-7B-Instruct`. Its role is to estimate how much context is required to answer the question effectively. The model outputs a number (capped due to VRAM limitations), which is then multiplied by a baseline value (in our case, 4). This determines the number of chunks retrieved from the database.

To improve robustness, few-shot prompting is used, providing the model with examples of questions paired with appropriate context-size decisions. This dynamic retrieval strategy is particularly useful in cases where the user requests a specific number of items, such as naming 5 courses. If the number of retrieved chunks were fixed (e.g., to 3), the model would lack sufficient context to satisfy such queries, as it would only have access to a limited set of courses.

Agent

To give our model extra utility, we implement a set of tools that the model can use. These tools are focused on the timetable and are used so the user can ask questions about a possible overlap in time-slots between different courses. Available tools are: `get_course_schedule`, `check_overlap` and `courses_on_day`. The functionality of each tool should be obvious to the reader by the tool name.

The agent is given a maximum of 3 iterations before it returns an answer. This is again, due to VRAM limitations, but it also turns out to be enough in most cases.

The model that answers the question is the same `Qwen2.5-7B-Instruct` model. This way we don't have to load two different models into memory and save on VRAM.

Conversational Memory

The system supports multi-turn dialogue by maintaining a conversation history as a list of question–answer pairs. This history is serialised to a JSON file after each turn and reloaded at the start of the next session, ensuring continuity across separate invocations of the pipeline. The full history is prepended to each new prompt so the model can resolve references to previous turns.

Results

Since course descriptions are often both in English and Slovenian, we evaluate the model in both languages. The same set of tests is run, where the input query is submitted once in Slovenian and once in English. To test the performance of our solution, we create four categories: **Simple Factual Queries** — these are designed to test the ability of the agent to answer simple questions such as naming courses that deal with a certain topic; **Comparison/Explanation Queries** — these queries mostly contain two courses and the agent is asked to explain the difference between them. Queries could also ask for comparison between different topics and not necessarily courses. **Timetable Queries** — the queries in this topic are focused on testing the agentic functionality of our chat-bot. All queries require that the time-table is checked by asking whether courses overlap, when lab sessions are, etc. **Complex Queries** — this is the set of the queries that test the reasoning of the model. They usually require the agent to extract specific information about multiple courses and reason about the answer.

Every question was also input into a commercial large language model Deep Seek. This gives us baseline performance we can compare to.

We compute three metrics for evaluation. Firstly, for every query, we ask a different commercial LLM to compare the answer to our ground-truth answer and rate it from 1 – 5, based on how relevant the answer is and how factual it is. Secondly, we embed the ground truth and the answer of each model and calculate the cosine similarity between them. Lastly, we evaluate each answer manually and rate the answers on a scale from 1 to 5. This is done by multiple people independently and the scores are averaged.

Figures 1 and 2 display the mentioned evaluation metrics for Slovenian and English respectively, grouped by query type. The average performance of the models across all categories is shown in Tables 1 and 2 for Slovenian and English respectively.

Looking at the results, our solution achieves performance comparable to the baseline DeepSeek model. However, it is important to note that the baseline model was not evaluated on the timetable-related category of questions, as it did not have access to the tools available to our agentic system and therefore could not reliably handle such queries.

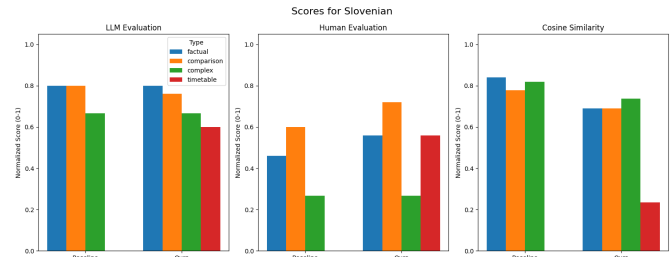


Figure 1. Normalized metric values for the baseline model and our model tested on Slovenian queries.

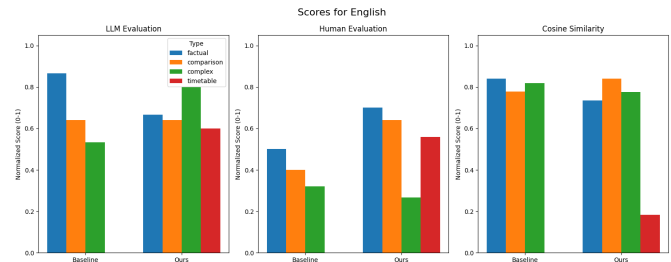


Figure 2. Normalized metric values for the baseline model and our model tested on English queries.

While our method shows a lower average cosine similarity score, we hypothesize that this is partly due to the agent frequently responding in a mix of Slovenian and English, which naturally reduces similarity-based evaluation scores. In contrast, DeepSeek is a more capable multilingual model and does not exhibit these limitations.

A manual review of the outputs further indicates that, in most cases where the agent produced irrelevant answers, the underlying issue was the absence of relevant information in the retrieved context. This suggests that the effectiveness of the RAG pipeline is the primary determining factor for performance in this task.

There is not a big difference between the quality of answer between using Slovenian and English language.

Summary

We presented a solution for a specialized agent for course querying. Our approach primarily focused on improving the quality of the RAG pipeline and the relevance of retrieved context, but was ultimately limited by the quality of the underlying dataset.

We evaluated the system in both English and Slovenian and found no noticeable difference in performance between the two languages. When compared to a commercial model such as DeepSeek, our solution achieved slightly lower results in both LLM-based and human evaluations.

Overall, these results suggest that while the architecture is sound, further gains are likely dependent on improvements in dataset quality and retrieval robustness.

Model \ Metric	Baseline	Ours
LLM eval.	3.78	3.79
Human eval.	2.85	2.78
Cosine distance	0.81	0.56

Table 1. Average performance of models on Slovenian queries.

Model \ Metric	Baseline	Ours
LLM eval.	4.57	3.42
Human eval.	3.13	2.92
Cosine distance	0.81	0.60

Table 2. Average performance of models on English queries.

Discussion

Overall, our system achieved performance comparable to a general-purpose commercial model on the task of extracting information about courses. We conclude that the quality and structure of the provided context are likely the most critical factors in building a usable chatbot, more so than model choice alone.

Future improvements should focus on enhanced data pre-processing and enriching each chunk with additional metadata to improve retrieval and response accuracy. However, this was constrained by the low quality of the dataset, and generating reliable metadata would have required extensive manual data cleaning and curation.

Finally, the performance of both the reasoning and embedding models was limited by the available GPU and VRAM, which restricted the size and capability of the models that could be used.

References

- [1] Bhavika R. Ranoliya, Nidhi Raghuwanshi, and Sanjay Singh. Chatbot for university related faqs. In *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 1525–1530. IEEE, 2017.
- [2] Fabio Clarizia, Francesco Colace, Marco Lombardi, Francesco Pascale, and Domenico Santaniello. Chatbot: An education support system for student. In *Cyberspace Safety and Security: 10th International Symposium, CSS 2018*, volume 11161 of *Lecture Notes in Computer Science*, pages 291–302, Cham, 2018. Springer.
- [3] Jakub Swacha and Michał Gracel. Retrieval-augmented generation (rag) chatbots for education: A survey of applications. *Applied Sciences*, 15(8):4234, 2025.
- [4] University of Ljubljana, Faculty of Computer and Information Science. Računalništvo in informatika (uni).

https://www.fri.uni-lj.si/upload/Zborniki/1000468_Ra%C4%8Dunalni%C5%A1tvo_in_informa%20-%20Copy%2014.pdf, n.d. Accessed: 2026-03-19.

- [5] University of Ljubljana, Faculty of Computer and Information Science. Računalništvo in informatika (vss). https://www.fri.uni-lj.si/upload/Zborniki/1000470_Ra%C4%8Dunalni%C5%A1tvo_in_informa%20-%20Copy%2013.pdf, n.d. Accessed: 2026-03-19.
- [6] University of Ljubljana, Faculty of Computer and Information Science. Urnik fri. https://urnik.fri.uni-lj.si/timetable/fri-2025_2026-letni/, n.d. Accessed: 2026-03-19.
- [7] University of Ljubljana, Faculty of Computer and Information Science. U1 fri. <https://www.fri.uni-lj.si/sl>, n.d. Accessed: 2026-03-19.